



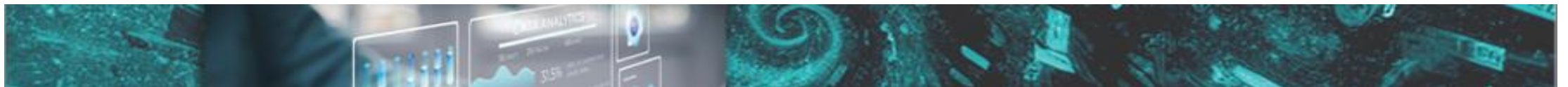
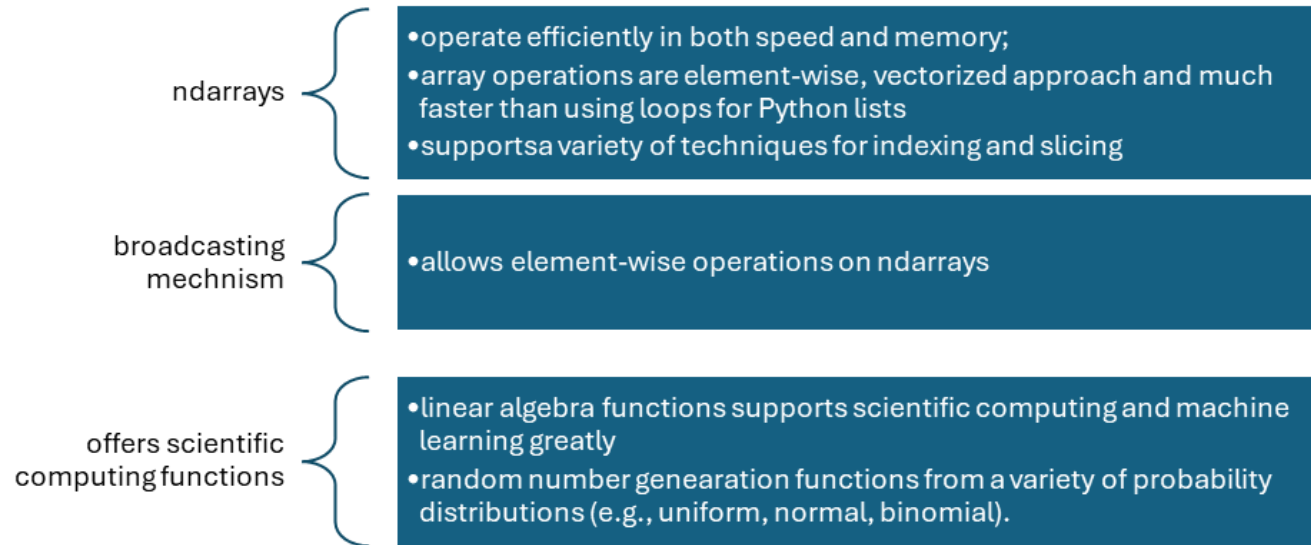
Chapter 2 Numpy

Data Analytics Using Python

Dr. Ziping Liu

Why NumPy

- NumPy, short for Numerical Python
- a Python package for scientific computing to handle large, multidimensional arrays and matrices efficiently
- core data structure: **ndarray** (n-dimensional array)
- restricted to a single data type
- compared to Python list, ndarrays are much **faster and use less memory**

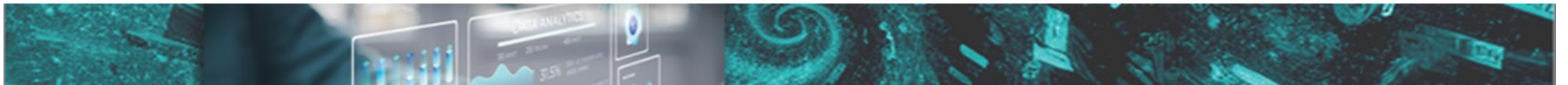


Installation and Testing

- Installation and testing

- If you only have Python installed on your system, you will need to install NumPy
- If you have Anaconda installed, you can check whether NumPy is already installed on your system

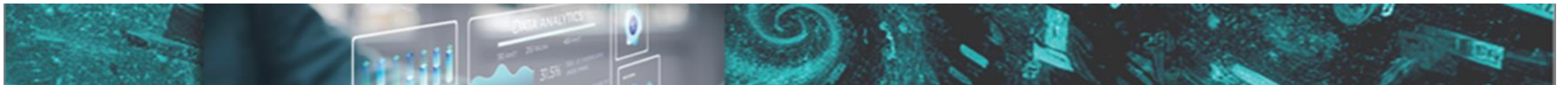
Using pip:	Using conda:
<code>pip install numpy</code>	<code>conda install numpy</code>
Verifying the installation: <code>import numpy as np</code> <code>print(np.__version__)</code>	
Testing the NumPy installation using a simple script: <code>import numpy as np</code> <code>arr = np.array([10, 20, 30, 40, 50])</code> <code>print(arr)</code>	



Getting Started with NumPy

- ndarray
 - can store and manipulate large datasets
 - comparison between NumPy ndarray and Python list

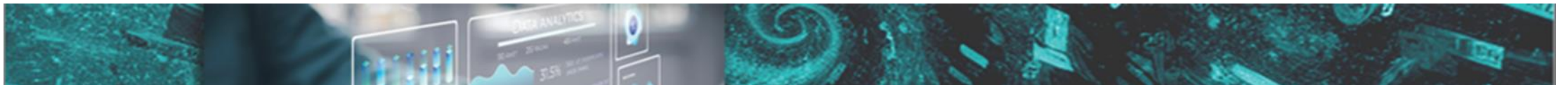
	NumPy ndarray:	Python list:
speed	faster with element-wise ops	slower with loops
data types	homogeneous	heterogenous
size	fixed once created	can be updated



Getting Started with NumPy

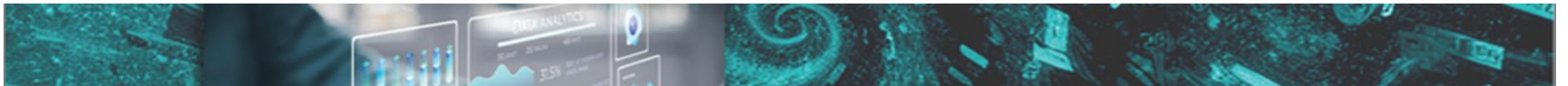
- Use NumPy and create ndarray
 - import the package: `import numpy as np`
 - create an ndarray: see examples in the textbook

function	description
array(sequence)	creates an array with an existing sequence passed in
random functions: rand(), randn(), normal()	generate arrays with random numbers
arange(start, end, exclusive, step)	returns an array having sequence of numbers with specified step and values are generated within the half-open interval [start, end)
linspace(start, end and number of items)	returns array having a specified number of elements
zeros()	creates a ndarray with all zeros
ones()	creates a ndarray with all ones



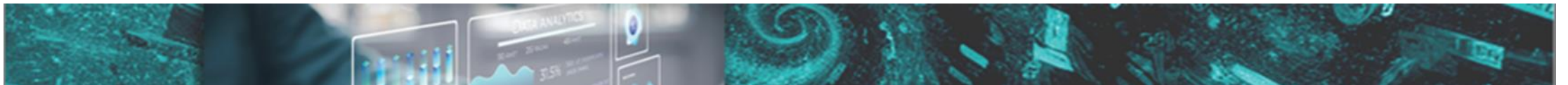
NumPy Basic Operations

- universal functions (ufuncs), optimized for element-wise operations: see examples in the textbook
 - Arithmetic operations: $+$, $-$, $*$, $/$, $\%$,
 - Mathematical operations: `np.add`, `np.subtract`, `np.multiply`, `np.divide`, `np.power`,
 - Trigonometric functions: `np.sin`, `np.cos`, `np.tan`,
 - Logarithmic functions: `log`, `log10`, `log2`, `log1p`,
 - Statistical functions: `np.mean`, `np.median`, `np.std`, `np.var`,
 - Comparisons: `==`, `!=`, `<`, `>`, `<=`, `>=`,
 - Logical operations: AND, OR, NOT



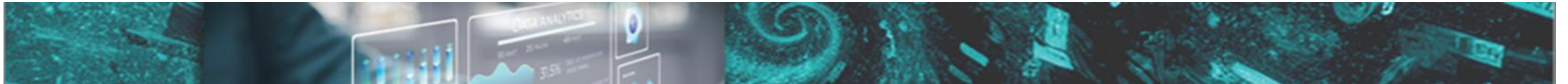
NumPy Basic Operations

- Vectorization
 - like mathematical vector operations, operations can also be applied over entire arrays simultaneously instead of using loops to process one element at a time
 - characterized by two key concepts: element-wise operations and broadcasting
 - vectorization is significantly faster than using loops: see textbook example
 - broadcasting in NumPy facilitates this by expanding the smaller array to match the shape of the larger one before executing the operation when operations involve arrays of different shapes: see textbook example



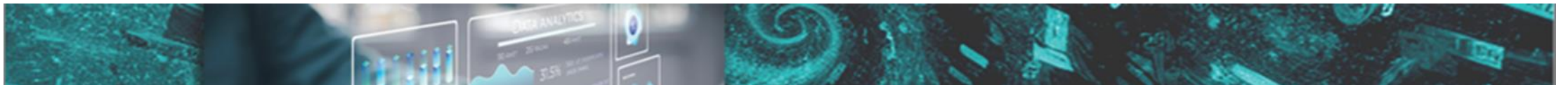
Indexing and Slicing

- Indexing: using a single index value or multiple index values, see textbook example
- Slicing: retrieving a sequence or block of elements from an array. The result of slicing is a shallow copy, which can be modified. see textbook example
- When indexing and slicing are applied to multi-dimensional arrays, various sections of the array can be accessed
- In slicing, the colon (:) retrieves all elements along a specific dimension, and the ellipsis (...) produces a complete indexing tuple with as many colons as needed
- Slicing can also be used to expand an existing array by inserting a new dimension. see textbook example



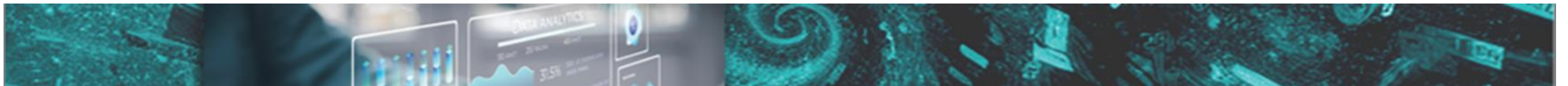
Indexing and Slicing

- Fancy indexing
 - use indexing with arrays of indices to select multiple elements from an axis: see textbook example
 - use an array of indices on a multidimensional array to create a new array by selecting specific rows or columns: see textbook example



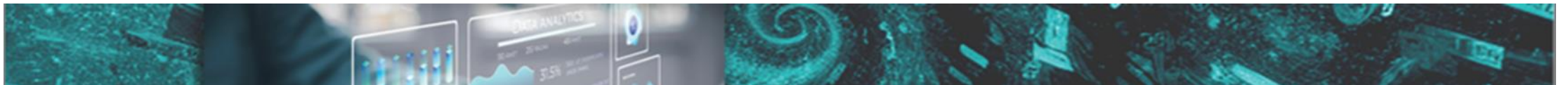
Indexing and Slicing

- Boolean indexing
 - Boolean arrays are arrays where elements have values of either True or False, and they can be used for indexing: see textbook example



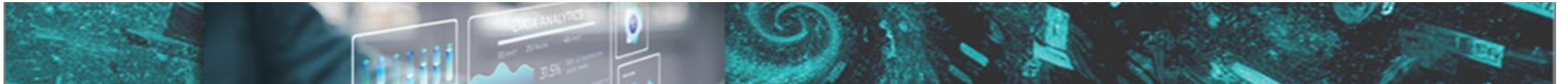
Various Operations

- `argmax()` and `argmin()` functions
 - `argmax()`: find the index of the maximum value(s) along a specified axis in an array, such as `axis = 0`, `axis = 1`, etc. See textbook example
 - `argmin()`: find the index of the minimum value(s) along a specified axis in an array, such as `axis = 0`, `axis = 1`, etc. See textbook example



Various Operations

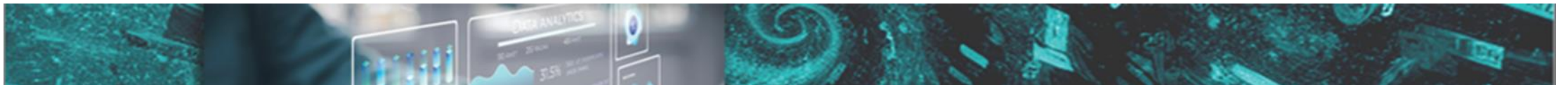
- operations along a specified axis: see textbook example
 - `sum()`
 - `mean()`
 - `cumsum()`
 - `sort()`
 - `argsort()`



Various Operations

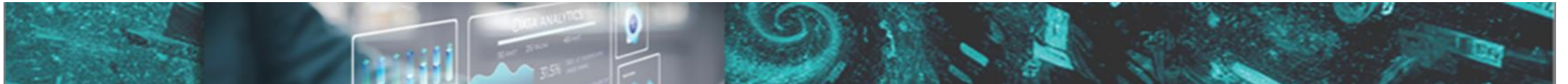
- reshape: change the dimension of an array either by creating a new view with the specified dimensions or by modifying the array itself. See textbook example

Function	Description
reshape()	parameter list: a tuple of integers or a single integer. return: a new view of the array, original array is not altered.
ravel()	parameter list: return: a flattened array of one dimension, the original array is not altered.
resize()	parameter list: a tuple of integers or a single integer. return: a new view of the array, original array is altered.
np.transpose()/T	parameter list: a tuple of integers for the permutation of the axes. If not provided, the default is to reverse the order of the axes. return: a new view of the array, original array is not altered.



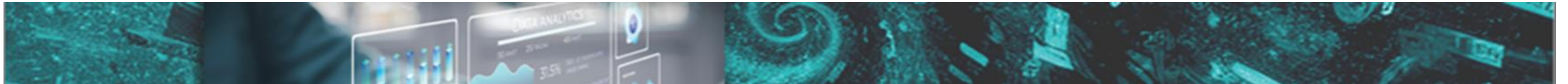
Various Operations

- Concatenating and Splitting Array: see textbook example
 - concatenate(): specify arrays and axis
 - split(): specify array and axis



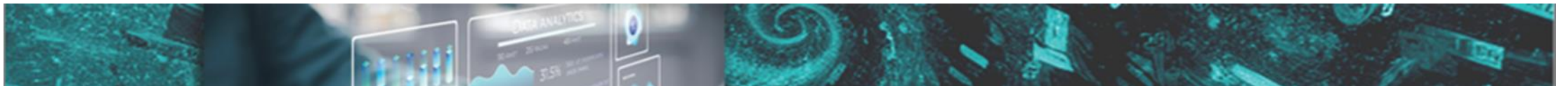
Various Operations

- Array data type conversion: see textbook example
 - determine the data type of a NumPy array using the dtype attribute
 - convert an array's data type from one dtype to another using NumPy's `astype()` method



Structured Array

- supports arrays where different columns can have different data types
- each column (field) can have a distinct data type, which is specified using a list of field names and their corresponding data types
- see textbook example



Importing Data from files in NumPy

- several methods to load data from text-based files, including .csv files
 - `genfromtxt()`,
 - `loadtxt()`,
 - `fromfile()`,
 - `load()`.
 - `genfromtxt()` is flexible in handling data with various data types and delimiters: see textbook example

